통합반 1회차 실습 1 — 학습노트 만들기

제작용 · 비낭독: 이론① 11장 뒤 진행한다. 아래 조작·입력·확인·오류 분기는 낭독하지 않는다. 발화는 강사가 직접 작업하며 말한다. 화면은 아직 캡처하지 않았으며 확인 문장은 해당 결과를 실제로 확인하면서 말한다. 이 대본에는 고정 실습 시간이 없다.

자료 다운로드와 작업 공간 열기

조작: 회차 안내의 실습 1 ZIP을 내려받고 압축을 푼다. VS Code의 파일 > 폴더 열기로 풀린 폴더를 연다. 학습노트_입력틀.md와 참고_교재_발췌.md를 확인한 뒤 Notion의 수업용 작업 공간을 연다.

이번에는 앞으로 배운 내용을 쌓아 둘 노트를 만들겠습니다. 먼저 실습 자료를 내려받아 압축을 풀고, 이 폴더를 VS Code에서 열겠습니다. 입력틀은 작성할 항목을 확인할 때 쓰고, 실제 기록은 Notion에 남기겠습니다.

자료에는 앞에서 본 지식 자산화 설명이 들어 있습니다. 첫 기록을 쓴 뒤 이 자료와 비교하겠습니다. 지금은 입력틀에서 어떤 자리가 필요한지만 보고 Notion으로 넘어갈게요.

과정 홈과 기록 목록 만들기

조작: Notion에 새 페이지를 만들고 제목을 내 학습노트로 정한다. 개념 정리·참고 자료·실습 결과의 세 구획을 만든다. 개념 정리 아래 데이터베이스를 만들고 분야·이해도·작성일 속성을 추가한다. 이해도 값은 설명 못함 / 대충 앎 / 설명 가능으로 둔다. 메뉴 이름은 설치된 판본에서 해당 기능을 찾아 사용한다.

과정 홈부터 만들겠습니다. 제목은 내 학습노트로 하고, 개념 정리와 참고 자료, 실습 결과를 나누어 놓겠습니다. 여기서 필요한 기록으로 들어갈 수 있게 만드는 겁니다.

개념 정리에는 기록 목록을 만듭니다. 한 줄을 열면 내용을 쓸 수 있는 페이지가 됩니다. 분야는 학습 방법으로 넣고, 이해도와 작성일도 붙이겠습니다. 나중에 같은 분야를 모아 보거나 아직 설명이 어려운 기록을 찾는 데 쓸 항목입니다.

기록 양식과 첫 설명 작성하기

조작: 데이터베이스에 새 템플릿을 만든다. 본문에 내 설명 / 참고 자료와 해당 부분 / 다음에 확인할 것을 넣는다. 기본 템플릿으로 지정한 뒤 새 기록을 만들고 양식이 적용되는지 확인한다.

기록할 때마다 같은 제목을 다시 적지 않도록 양식을 만들겠습니다. 내 설명, 참고 자료와 해당 부분, 다음에 확인할 것을 넣습니다. 이제 이 양식으로 새 기록을 열어 보겠습니다.

첫 제목은 지식 자산화입니다. 자료를 잠깐 덮고 이렇게 써 볼게요. 배운 내용을 내 말로 남기고, 출처와 해 본 일을 연결해서 나중에 다시 확인하고 쓸 수 있게 하는 것입니다.

입력: 첫 설명은 위 발화를 직접 입력한다. 참고 자료 구획에 교재 이름과 5~9장 위치를 적고 제공된 발췌 파일을 첨부한다. 개념 기록에서 그 참고 자료로 연결한다.

이제 자료를 다시 보겠습니다. 제 설명에는 나중에 찾아 쓴다는 목적이 들어 있습니다. 다만 내 말로 썼다는 이유만으로 정확해지는 것은 아니죠. 설명이 맞는지 출처와 비교하고, 잘못 이해한 부분은 고쳐야 한다는 문장을 보태겠습니다.

참고 자료에는 문서 이름만 적지 않고 어느 부분을 봤는지도 남깁니다. 이 개념 기록에서 자료를 열어 같은 설명으로 돌아갈 수 있게 연결하겠습니다.

실습 결과 연결하고 다시 찾기

조작: 실습 결과 구획에 학습노트 만들기 기록을 만든다. 작업·확인 결과·남은 점을 적고 지식 자산화 개념 기록에 연결한다. 이해도가 설명 가능이 아닌 기록을 모으는 재학습 보기를 만든다. 과정 홈으로 돌아가 기록을 검색하고 첨부한 출처를 다시 연다.

실습 결과에는 방금 한 일을 남기겠습니다. 과정 홈과 목록, 양식을 만들었고, 첫 설명을 출처와 비교해 고쳤다고 적습니다. 개념 설명은 이미 있으니 다시 복사하지 않고 그 기록을 연결합니다.

이해도는 지금 자료 없이 설명할 수 있는 범위에 맞춰 고릅니다. 아직 남은 질문이 있다면 다음에 확인할 것에 적고, 재학습 보기에 그 기록이 나오는지 확인하겠습니다. 보여 주기 위해 일부러 이해도를 낮추지는 않겠습니다.

마지막으로 과정 홈으로 돌아가 지식 자산화 기록을 찾습니다. 여기서 참고 자료를 열고, 실습 결과에서도 같은 개념으로 돌아가 보겠습니다. 기록을 다시 찾고 근거를 확인할 수 있으면 이번에 만들려던 노트의 연결을 갖춘 겁니다. 다음에는 시스템 구조를 배우면서 이 노트에 각 부분의 역할을 남기겠습니다.

확인: 새 기록에 양식이 들어가는가 / 분야·이해도로 찾을 수 있는가 / 개념에서 출처를 열 수 있는가 / 실습 기록에서 개념으로 돌아갈 수 있는가.

오류 분기 · 해당할 때만: 새 페이지가 비어 있으면 기본 템플릿의 적용 보기를 확인한다. 링크가 다른 페이지로 열리면 실제 참고 자료 페이지 링크를 다시 연결한다. 확인하지 못한 항목은 미확인으로 남긴다. 공개 게시·학생 제출 대기·불필요한 꾸미기는 진행 조건으로 두지 않는다.

통합반 1회차 실습 2 — 실행 환경과 서비스 연결 확인

제작용 · 비낭독: 이론② 45장 뒤 첫 실습. 실제 시스템 실행은 이번 문서 작업에서 하지 않았다. 준비된 수업용 HYD 프로젝트·설정·인증이 필요하다. 실습 ZIP은 확인표와 안내 자료이며 시스템 전체가 아니다. 기동 판본·프로필·워커 연결은 안내의 사전 조건을 확인한다. 아래 명령은 강의 때의 조작 지시이며 지금 실행한 기록이 아니다.

자료 다운로드와 프로젝트 열기

조작: 실습 2 ZIP을 내려받아 압축 해제하고 VS Code로 폴더를 연다. 환경_준비_안내.md와 환경_확인표.md를 읽는다. 준비된 HYD 프로젝트를 별도 VS Code 창에서 폴더로 연다. 프로젝트 루트의 compose.yaml과 ot·dmz·it 폴더를 확인한다. 설정의 비밀값은 화면에 노출하지 않는다.

앞에서 본 데이터와 명령의 경로를 이제 실행 환경에서 확인하겠습니다. 자료를 내려받아 폴더를 열고, 확인표를 준비할게요. 이 자료 폴더에는 기록할 틀이 있고, 실행할 프로그램은 별도로 준비한 HYD 프로젝트에 있습니다.

프로젝트를 열면 현장 쪽, 중간 연결 쪽, 업무 쪽 폴더가 보입니다. 서비스가 어떻게 연결되는지는 Compose 파일에 정의돼 있습니다. 오늘은 이 구성을 실행하고, 켜졌다는 표시와 실제 연결을 차례로 확인하겠습니다.

도구와 배포 설정 확인

조작: 프로젝트 루트에서 Git Bash 터미널을 연다. Docker Desktop 엔진 상태를 확인한다. docker --version, docker compose version, python --version을 차례로 확인하고 docker info로 엔진에 닿는지 확인한다. 환경 안내에 적힌 프로필과 수업용 설정을 대조한다.

터미널에서 도구가 준비됐는지 확인합니다. 버전이 나온다는 것은 명령을 찾았다는 뜻입니다. Docker는 엔진에도 연결돼야 컨테이너를 실행할 수 있으니 여기까지 확인하겠습니다.

Python은 자체 서비스와 랩 코드에 쓰입니다. 브로커와 데이터베이스는 각자의 프로그램이고, Docker가 실행 환경을 묶어 줍니다. 도구 역할은 여기서 구분하고, 이제 실제 서비스를 켜겠습니다.

업무 저장소와 서비스 기동

입력 · Git Bash, 프로젝트 루트: (cd it/supabase && supabase start)

업무 저장소부터 켜겠습니다. 사건과 승인, 작업지시를 남기는 서비스가 이 저장소에 연결됩니다. 준비 상태가 확인되면 나머지 서비스를 올립니다.

입력 · Git Bash: COMPOSE_PROFILES=ot,backbone,detect,knowledge,agent,enterprise,process,cliagents docker compose up -d --scale agent-worker=0

조작: 준비 안내와 compose.yaml에서 해당 판본의 프로필·호스트 워커 사용을 확인한 뒤 위 명령을 적용한다. 처음 빌드가 필요한 배포본은 해당 배포 안내를 따른다. 호스트 워커는 이미 동작 중인지 먼저 확인하고 없을 때만 별도 Git Bash에서 bash scripts/run_worker_host.sh를 실행한다. 한 개만 유지한다.

여러 서비스가 올라오는 동안 상태를 보겠습니다. 실행 순서를 외우기보다는 어느 의존 서비스가 준비됐는지를 확인합니다. 한 번 자료를 넣고 끝나는 준비 작업도 있으니 종료됐다는 표시를 모두 실패로 읽지는 않겠습니다. 종료 코드와 그 작업의 역할을 함께 보겠습니다.

에이전트 작업을 처리할 워커도 확인합니다. 포털이 열려도 워커가 연결되지 않으면 다음 실습의 진단 단계가 진행되지 않을 수 있습니다. 이미 실행 중인 워커가 있으면 하나를 더 띄우지 않고 그 상태를 확인하겠습니다.

상태와 접속 결과를 확인표에 남기기

입력 · Git Bash: docker compose ps -a; (cd it/supabase && supabase status); curl -s http://127.0.0.1:8080/healthz; curl -s http://127.0.0.1:8097/health

조작: 상시 서비스는 실행 상태와 제공되는 healthcheck를, 초기화 작업은 종료 코드와 로그를 확인한다. 포털 http://127.0.0.1:8088을 열어 연결 상태와 HYD-01 계측 갱신을 확인한다. 실제 주소가 다른 배포본은 안내의 주소를 쓴다. 확인표에는 실제로 본 서비스·상태·확인 시각·연결 결과를 적는다.

이제 목록을 확인표와 대조하겠습니다. 개수가 맞아도 필요한 서비스 하나가 빠졌을 수 있습니다. 계속 떠 있어야 하는 서비스와 한 번 실행하고 끝나는 작업을 나눠 확인합니다.

프로세스의 응답과 워커 상태를 보고 포털을 열겠습니다. 화면이 열리는 것까지 확인한 뒤, 설비 값이 갱신되는지도 봅니다. 단순히 페이지가 보이는 것과 설비에서 값이 올라오는 것은 다른 확인입니다.

확인표에는 지금 화면에서 확인한 결과를 적습니다. 접속이 안 되는 곳은 정상이라고 채우지 않고, 오류 내용과 마지막으로 확인된 지점을 남기겠습니다.

오류 분기 · 해당할 때만: 엔진 연결 실패는 Docker Desktop 상태부터, 업무 DB 연결 실패는 Supabase 상태와 포트부터, 진단 대기는 워커 연결과 배포 설정부터 확인한다. 필요한 서비스의 로그만 읽고 원인을 좁힌다. 볼륨 삭제나 전체 초기화로 해결하려 하지 않는다. 해결되지 않으면 다음 사건 실습의 선행 조건 미충족으로 기록한다.

다음 실습으로 이어가기

확인표에서 서비스 실행, 포털 접속, 계측 갱신과 워커 연결을 대조하겠습니다. 준비된 항목과 남은 문제가 구분됐습니다. 이 환경을 유지한 채 다음 실습에서 쿨러 사건 하나를 관찰하고 기록을 만들겠습니다.

확인: 위 선행 조건이 충족될 때만 실습 3으로 간다. 끝난 뒤 종료는 배포 안내의 워커·서비스·업무 저장소 순서를 따른다. 이 사이에는 끄지 않는다.

통합반 1회차 실습 3 — 쿨러 사건 관찰과 근거 연결표 만들기

제작용 · 비낭독: 실습 2의 환경을 유지한다. 수업용 가상 설비 한 대의 사건 한 건만 관찰한다. 현재 사건의 ID·값·소요 시간은 실제 화면에서 기록한다. 교재 예시의 숫자를 새 실행 결과로 읽지 않는다. 아직 전체 사건을 실행하거나 캡처하지 않았다. 아래 확인 발화는 관찰 결과가 맞을 때만 사용한다.

자료 다운로드와 기록할 자리 열기

조작: 실습 3 ZIP을 내려받아 압축을 풀고 VS Code에서 연다. 근거_연결표.md와 관찰_기록.md를 연다. 앞 실습의 포털과 워커 연결 상태를 확인한다. 다른 진행 중 사건은 초기화하지 않는다.

이번에는 사건을 보면서 근거 연결표를 직접 만들겠습니다. 자료를 내려받아 폴더를 열면 빈 표가 있습니다. 화면에 나온 번호와 참조를 여기에 적고, 나중에 같은 기록을 다시 찾는 데 쓰겠습니다.

포털은 앞에서 준비한 환경을 이어 씁니다. 먼저 관찰할 설비와 현재 상태를 기록합니다. 다른 사건과 섞이지 않도록 이번에 생기는 사건 번호도 함께 남기겠습니다.

경보에서 사건 찾기

조작: 포털에서 운전원 역할을 선택한다. 준비된 HYD-01의 정상 상태를 확인하고 쿨러 열화를 주입한다. 유온·냉각 효율·경보 상태를 관찰한다. 생성된 경보와 처리 건을 열어 설비·시각·식별자를 기록한다. 시간 배율은 수업 설정 20배를 유지한다.

쿨러의 냉각 성능을 낮춰 보겠습니다. 냉각 효율과 유온이 어떻게 바뀌는지 보고, 경보가 생기면 그 기록을 열겠습니다. 앞에서 배운 복합 조건이 성립했을 때 감지 서비스가 남기는 기록입니다.

경보에 남은 값은 발생 당시의 값입니다. 옆에서 계속 바뀌는 현재 값과 나누어 적겠습니다. 이어서 이 경보를 처리하는 사건을 찾고, 사건이 어느 경보를 참조하는지 표에 옮깁니다.

진단과 추천의 근거 확인

조작: 처리 건의 완료된 진단·추천 단계를 연다. 원인 후보, 조회한 태그와 구간, 판별 기준, 실제 값, 선택한 조치와 대안을 읽는다. 관찰 기록에 실제 결과와 근거 위치를 적는다. 진행 중에는 처리 상태만 확인하고 같은 개념을 반복해 시간을 채우지 않는다.

진단 결과를 열겠습니다. 결론만 옮기지 않고 무엇을 읽고 어떤 기준과 비교했는지 함께 적습니다. 경보 당시의 유온과 원인을 가릴 때 사용한 평균값은 쓰임이 다르므로 칸을 나누겠습니다.

추천에는 고른 조치뿐 아니라 다른 대안도 있는지 보겠습니다. 냉각 효과와 생산 일정이 선택에 어떻게 반영됐는지, 기록에 남은 근거를 확인합니다. 조회가 성공했다는 것만으로 결론까지 맞다고 넘기지 않겠습니다.

오류·차이 분기: 근거 조회가 실패하거나 자료가 부족하면 해당 항목을 미확인으로 남기고 다음 단계의 승인으로 넘기지 않는다. 교재와 다른 원인·조치가 나오면 관측값과 판별 기준부터 비교한다. 기대한 결론이 나올 때까지 같은 사건을 반복하지 않는다.

승인과 명령 처리 확인

조작: 추천안 검토 화면에서 현재 운전 상태를 다시 확인한다. 승인 전에는 사건의 명령 기록이 아직 없는지 확인한다. 추천 근거와 실행 조건이 맞는 경우에만 운전원으로 승인한다. 승인 기록·명령 ID·PLC ACK를 찾아 관찰 기록에 남긴다.

이제 승인할 차례입니다. 추천을 만드는 동안에도 설비 상태는 달라질 수 있으니 현재 상태를 다시 보겠습니다. 승인 전에는 이 사건의 명령 기록이 있는지 확인합니다. 팬 속도가 그대로라는 사실만으로 명령이 없다고 단정하지 않고 기록을 직접 보겠습니다.

근거와 현재 조건을 확인하고 승인합니다. 그다음 생성된 명령과 응답의 번호를 맞춰 보겠습니다. 같은 명령에 대한 응답인지 확인해야 처리 결과를 연결할 수 있습니다.

분기: 승인 조건이 맞지 않으면 거절·보류하고 이유를 기록한다. ACK가 없거나 거절 응답이면 그 결과를 적고 성공 발화를 하지 않는다. 사용자가 선택하지 않은 조치를 자동으로 승인한 것으로 설명하지 않는다.

완화와 정비 작업 구별하기

조작: 재관측 단계의 유온·경보 상태·판정 구간을 확인한다. 핀 세척 작업지시의 존재와 현재 진행 상태를 별도로 읽는다. 미완료 작업지시를 완료로 바꾸거나 설비 복구 버튼을 정비 완료의 증거로 삼지 않는다.

명령 처리 뒤에는 재관측 결과를 보겠습니다. 응답이 완료라고 해도 원하는 냉각 효과가 있었는지는 이 기록에서 따로 확인합니다. 유온과 경보 상태가 정해진 회복 조건을 만족했는지 보겠습니다.

핀 세척 작업지시도 열겠습니다. 팬 운전으로 급한 유온을 낮춘 것과 핀을 세척한 것은 다른 일이죠. 작업지시가 등록됐는지, 승인됐는지, 실제 수행이 끝났는지를 현재 기록대로 구분해 적겠습니다.

작업지시에서 근거로 거슬러 가기

조작: 작업지시의 판단 참조, 판단의 사건 참조, 사건의 경보 참조를 차례로 읽어 근거_연결표.md를 채운다. 각 행에는 출발 기록 ID / 참조 항목 이름 / 참조 값 / 도착 기록 ID / 확인 위치를 적는다. 다시 원기록을 열어 참조 값과 도착 ID를 대조한다.

이번에는 작업지시에서 거꾸로 가 보겠습니다. 어떤 판단을 참조하는지 찾아 그 값을 표에 적고, 같은 번호의 판단 기록을 엽니다. 이어서 그 판단의 사건, 사건의 경보까지 같은 방법으로 연결합니다.

이름이 비슷한 기록을 고르는 것이 아니라 참조 값을 맞춥니다. 화면에 참조 항목이 없다면 해당 기록의 상세 조회 자료에서 확인하고, 거기에도 없으면 그 연결은 미확인으로 두겠습니다.

표를 채웠으니 작업지시부터 한 번 다시 열어 보겠습니다. 각 참조가 실제 다음 기록을 가리키는지 대조합니다. 연결표 옆에는 경보 당시 값과 원인 판별 증거를 구분해 놓았습니다. 이제 조치의 이유를 묻는다면 이 기록을 따라가 확인할 수 있습니다.

확인: 같은 사건의 기록인가 / 참조 값과 도착 ID가 일치하는가 / 명령 처리·완화·정비 상태를 분리했는가 / 없는 근거를 추정해 채우지 않았는가. 필요한 경우 해당 설비만 수업 운영 절차에 따라 복구한다. 전체 초기화는 하지 않는다.

관찰 기록과 연결표를 저장하겠습니다. 다음 이론에서는 이렇게 원인과 조치를 판단하는 데 사용한 지식이 어떤 구조로 연결돼 있는지 살펴보겠습니다.

통합반 1회차 실습 4 — 두 원인을 비교하는 관계 그림

제작용 · 비낭독: 이론③ 71장 뒤 진행하고 72장으로 복귀한다. 실제 arrows.app 조작·AI 대화·화면 캡처는 수행하지 않았다. 아래 내용은 강사가 만들고 확인할 작업 대본이다. 고정 노드 수나 문답 횟수를 목표로 두지 않는다.

자료 다운로드와 작업 공간 열기

조작: 실습 4 ZIP을 내려받고 압축을 푼 뒤 VS Code에서 폴더를 연다. 대상_목록.md, 매뉴얼_발췌.md, 관계_검토표.md를 연다. 브라우저에서 arrows.app을 열어 새 그림을 만든다.

이번에는 핀 오염과 팬 구동 불량을 비교할 관계 그림을 만들겠습니다. 자료를 내려받아 폴더를 열고, 대상 목록과 매뉴얼을 옆에 놓겠습니다. 그림은 arrows.app에서 만들고, 검토한 근거는 이 폴더에 남기겠습니다.

설비와 증상에서 출발하기

조작: HYD-01 유압 파워팩, 오일 쿨러, 유온 센서, 유온 상승, 쿨러 냉각 성능 상실 노드를 만든다. 각 노드에 종류를 표시한다. 노드 생성·이름 입력·관계 연결을 실제 그림을 만들면서 보여 준다. 연습용 그림을 별도로 만들었다 지우지 않는다.

설비와 오일 쿨러부터 놓겠습니다. 유온 센서와 유온 상승도 추가하고, 가운데에는 냉각 성능 상실을 놓습니다. 센서는 값을 읽는 대상이고 증상은 관측한 현상이므로 서로 다른 노드입니다.

설비가 오일 쿨러를 구성 요소로 가진다는 연결을 만듭니다. 오일 쿨러는 유온 센서가 계측하고, 유온 상승은 그 센서에서 관측됩니다. 유온 상승이 냉각 성능 상실을 가리키도록 연결하겠습니다. 이름과 방향은 방금 읽은 문장에 맞춥니다.

핀 오염의 증거와 조치 연결하기

조작: 핀 오염 노드에서 냉각 성능 상실로 CAUSES 관계를 연결한다. 오일 쿨러와 고장 유형 사이에는 OCCURS_IN을 연결한다. 핀 오염에 EVIDENCED_BY로 냉각 효율 30초 평균과 팬 진동 2분 평균의 증거 노드를 연결하고 기준을 속성으로 적는다. 매뉴얼 발췌 HM-8.4와 비교한다.

핀 오염은 냉각 성능 상실을 일으키므로 원인 쪽에서 화살표를 긋겠습니다. 그리고 핀 오염을 판단할 때 확인한 증거를 붙입니다. 냉각 효율은 최근 30초 평균, 팬 진동은 최근 2분 평균입니다. 두 값을 어떤 기준과 비교하는지도 매뉴얼에서 확인해 적겠습니다.

여기에는 판별 기준을 넣습니다. 지난 실습에서 관찰한 값은 그 사건의 기록에 있고, 이 그림은 다음 사건에서도 어떤 값을 확인할지 알려 주는 구조입니다.

조작: 팬 최대 운전과 핀 세척 노드를 추가한다. 고장 유형에서 MITIGATED_BY와 REMEDIED_BY를 구분해 연결한다. 핀 세척에는 핀 오염 확인 시 적용한다는 조건과 HM-8.7 근거를 표시한다. 근거 표기의 주석은 시스템에 저장된 새 관계라고 설명하지 않는다.

팬 최대 운전은 유온을 우선 낮추는 조치로 연결합니다. 핀 세척은 오염을 없애는 조치로 따로 놓겠습니다. 같은 고장 유형에서 이어지더라도 모든 원인에 세척을 적용하는 것은 아니므로, 핀 오염을 확인했을 때라는 조건과 매뉴얼 근거를 함께 적습니다.

팬 구동 불량 가지 추가하기

조작: 팬 구동 불량 원인과 팬 점검 조치를 추가한다. 동일한 고장 유형으로 CAUSES를 연결한다. 팬 진동 2분 평균 증거는 재사용하되 원인별 판별 조건을 구별한다. 팬 점검의 근거 HM-8.1을 표시한다. 근본 조치 후보의 적용 원인을 주석으로 명시한다.

이제 팬 구동 불량을 더하겠습니다. 고장 유형은 그대로 두고 원인 가지를 하나 늘립니다. 팬 진동 평균은 앞 가지에서도 사용했지만 비교 조건이 다릅니다. 핀 오염에서는 정상 범위인지 보고, 팬 구동 불량에서는 이상 기준 이상인지 봅니다. 같은 증거를 쓴다고 두 원인을 합치지는 않겠습니다.

팬 구동 불량 쪽에는 매뉴얼의 팬 점검을 연결합니다. 핀 세척으로 이어 놓으면 손볼 대상이 달라집니다. 또 팬 자체가 이상한 경우에는 팬 최대 운전을 자동으로 선택하지 않도록, 원인과 실행 조건을 확인해야 한다는 주석을 남기겠습니다.

문장과 원문으로 대조하기

조작: 관계 검토표에 출발 대상 / 관계 이름 / 도착 대상 / 읽은 문장 / 원문 근거를 채운다. 원인 두 가지에서 증거와 조치로 이어지는 경로를 각각 검토한다. 실제 방향·이름 오류가 있을 때만 고친다.

그린 관계를 문장으로 읽겠습니다. 핀 오염은 냉각 성능 상실을 일으킨다. 팬 구동 불량도 같은 고장 유형을 일으킨다. 두 원인은 확인할 조건과 손볼 곳이 다릅니다.

문장이 자연스러운지만 보지 않고 대상의 종류와 매뉴얼 내용도 대조합니다. 고장 유형에서 원인 후보를 찾을 때는 들어오는 화살표를 거슬러 보면 됩니다. 찾는 방향이 반대라고 저장한 화살표를 뒤집을 필요는 없습니다.

선택 분기 · 스스로 판단이 어려울 때만 Copilot에 입력: "이 관계를 문장으로 읽으면 [내가 읽은 뜻]이고 원문의 [해당 부분]은 이렇게 설명합니다. 두 뜻이 어긋나는지 확인할 관점을 짚어 주세요. 완성 그림이나 정답 관계 목록 대신 제가 다시 비교할 부분을 알려 주세요." 실제 관찰 내용을 괄호 자리에 넣는다. 이미 맞게 연결한 관계를 다시 묻지 않는다.

저장하고 다시 열기

조작: 편집 가능한 그림 데이터를 내보내 관계그림.json으로 저장하고, 열람용 PNG도 저장한다. 새 그림에서 JSON을 다시 불러와 이름·방향·원인별 조건·근거 주석이 남아 있는지 대조한다. 원본을 덮어쓰기 전에 재열기 결과를 확인한다.

편집할 수 있는 데이터와 볼 수 있는 그림을 함께 저장하겠습니다. 새 그림에서 데이터를 다시 불러와 두 원인의 가지가 남아 있는지 확인합니다. 지금 저장한 그림은 다음 회차에 관계를 더 구체적으로 정리할 출발점이 됩니다.

관계 검토표도 저장하겠습니다. 핀 오염과 팬 구동 불량에서 각각 어떤 증거를 확인하고 어떤 조치로 이어지는지 읽을 수 있게 됐습니다. 이제 교재로 돌아가 사건 기록의 역할을 정리하겠습니다.

오류 분기 · 해당할 때만: 내보낸 파일이 없으면 다운로드 위치부터 확인한다. 재열기 후 속성이나 주석이 빠지면 원본 그림과 대조해 보완하고 다시 저장한다. 편집 데이터의 복원이 확인되지 않으면 완료로 표시하지 않는다.